

Sandbox, Safety-Aware Scheduling, and Audit

Lecture 6: HCR Simulator as Gatekeeper

Supervisor - Prof. Dr. Mohammed Aquil Mirza, COMP

Student Mentors - Mr. Ruan Haozhe, Mr. Jiang Suyu, Mr. Zhao Jiuyuan

June 2026

The Hong Kong Polytechnic University



Five-Constraint Stack

Sandbox input: what the checker sees

Candidate plan.

A cut plan names a robot, a tool, a path, timing, and target hair-length changes. It is proposed, not trusted.

Twin state.

The digital twin supplies scalp mesh, hair field, tool pose, robot reach, head pose, and predicted next state.

Sensor evidence.

The sandbox records the sensor timestamp and confidence used for the decision. Stale or missing critical evidence cannot silently pass.

Threshold version.

Desktop and Web load the same threshold version; private constants are drift.

The five constraints

Skin distance.

The shortest predicted distance from tool cutting surface to scalp skin must remain above the safety margin.

No-cut zone.

The planned path must not enter protected regions such as ears, eyes, moles, wounds, or user-marked exclusion zones.

Tool angle.

The blade or guard angle must stay in a range where the tool removes hair rather than scraping or poking skin.

The five constraints (Cont'd)

Hair-length floor.

The predicted post-cut hair field must not fall below the chosen length floor for the active style region.

Head-pose tolerance.

The user's actual head pose must remain close enough to the pose used by the simulated plan.

Constraint stack order

Run cheap structural checks first.

Reject malformed plans, unknown tool ids, impossible timestamps, and missing robot ids before running geometry.

Then run geometry checks.

Evaluate no-cut zones, skin distance, tool angle, and hair-length floor against the same predicted trajectory.

Then run live-state checks.

Evaluate head pose and freshness against the newest observed user state.

Return the worst state.

No averaging: reject beats confirm, and confirm beats approve.

Draft threshold table, geometry

The values below are teaching thresholds for the simulator fixture. If the released C3 contract table differs, use the released table.

Constraint	Approve	Confirm	Reject
Skin distance	$d \geq 8 \text{ mm}$	$4 \leq d < 8 \text{ mm}$	$d < 4 \text{ mm}$
No-cut margin	$m \geq 5 \text{ mm}$	$2 \leq m < 5 \text{ mm}$	intersects zone or $m < 2 \text{ mm}$
Tool angle	$20^\circ \leq a \leq 50^\circ$	$15^\circ \leq a < 20^\circ$ or $50^\circ < a \leq 60^\circ$	$a < 15^\circ$ or $a > 60^\circ$

Draft threshold table, state

The table is stated in physical units so Rust and TypeScript implementations normalize from the same source.

Constraint	Approve	Confirm	Reject
Hair-length floor	$l \geq f + 2 \text{ mm}$	$f \leq l < f + 2 \text{ mm}$	$l < f$
Head rotation	$\Delta r \leq 3^\circ$	$3^\circ < \Delta r \leq 7^\circ$	$\Delta r > 7^\circ$
Head translation	$\Delta t \leq 2 \text{ mm}$	$2 < \Delta t \leq 5 \text{ mm}$	$\Delta t > 5 \text{ mm}$

Freshness rule.

Critical sensor evidence older than 200 ms, or missing entirely, is a reject. Evidence between 80 ms and 200 ms is confirm.

Pseudocode: validate first

```
fn evaluate_plan(input: SandboxInput, t: Thresholds) -> DecisionLog {  
  if !input.schema_ok() {  
    return DecisionLog::reject("bad_input");  
  }  
  if input.critical_sensor_missing() {  
    return DecisionLog::reject("missing_sensor");  
  }  
  if input.sensor_age_ms > 200 {  
    return DecisionLog::reject("stale_sensor");  
  }  
}
```

Fail closed.

Malformed input, missing critical sensors, and stale critical sensors reject before geometry runs.

Pseudocode: run the stack

```
fn run_stack(input: SandboxInput, t: Thresholds) -> DecisionLog {  
  
    let checks = [  
        check_no_cut_zone(&input, &t),  
        check_skin_distance(&input, &t),  
        check_tool_angle(&input, &t),  
        check_hair_floor(&input, &t),  
        check_head_pose(&input, &t),  
    ];  
  
    DecisionLog::from_worst_state(checks)  
}
```

Why order does not change meaning

The stack may short-circuit for speed.

An implementation may stop after a hard reject if it still records which check caused the stop.

The fixture should also support full evaluation.

For reports and debugging, teams should be able to run all five checks and list every failing constraint.

Decision semantics are order independent.

The final state is the maximum under the order $\text{reject} > \text{confirm} > \text{approve}$. That rule is the same on both tracks.

Worked examples for the stack

Happy path.

All five constraints approve. The sandbox emits an approval with the threshold version and input hash.

Skin-distance violation.

The plan is otherwise valid, but predicted clearance is 3.5 mm. The whole plan rejects, even if scheduler pressure is high.

Worked examples for the stack (Cont'd)

Ambiguous head pose.

The path is safe, but head rotation drift is 5 degrees. The plan goes to confirm, not actuation.

Sensor impairment.

A critical sensor stream is missing. The sandbox rejects because the system cannot prove the safe condition.



Approve, Reject, Confirm

Three states

Approve.

The plan satisfies every constraint and all critical evidence is fresh. The robot may execute the approved message, not the raw plan.

Reject.

The plan violates a hard constraint, is malformed, uses stale critical evidence, or cannot be evaluated. No actuation message is emitted.

Confirm.

The plan is not proven unsafe, but it sits in an ambiguity band. A human confirmation or a safer replanning step is required before action.

Fail-closed principle

Default state is not approve.

If the sandbox cannot complete, cannot parse, cannot load thresholds, or cannot verify sensor freshness, the decision is reject.

Confirm is not a loophole.

Confirm is used for bounded ambiguity where the system can still show the operator what is uncertain. It is not used for missing proof.

Reject beats user intent.

A user, operator, or model may request a cut. The sandbox still rejects if the executable plan violates a constraint.

Decision-to-action mapping

Decision	Immediate action	Audit requirement
Approve	Publish approved plan to the actuation topic.	Record all five constraint results plus threshold version.
Confirm	Freeze actuation and request operator confirmation.	Record ambiguity band, operator prompt id, and timeout.
Reject	Do not publish actuation command.	Record reject reason, failed constraint, and input hash.

One rule.

The robot consumes only the approved plan topic. It never consumes raw VLA output.

Common implementation mistakes

Mistake 1: treating confirm as approve with a warning.

Confirm must stop robot action until the required follow-up arrives.

Mistake 2: using local thresholds.

Thresholds come from the C3 contract table. Private constants cause Desktop and Web drift.

Mistake 3: logging only failures.

Audit replay needs approved checks too, because later safety reports compare full decision evidence.

Mistake 4: ignoring missing data.

A missing sensor stream is not neutral. For critical evidence it is reject.



Safety-Aware Scheduling

Why scheduling is safety relevant

Scheduling changes what waits.

If every request is FIFO, a safety-critical confirm or near-skin replanning task can wait behind low-risk rendering or background work.

Priority must be explainable.

Every score is derived from five factors. The audit log records the factor values and the final integer score.

Priority must be byte-identical.

Ch5 and Ch6 implement the same pure function, with integer arithmetic and the same conformance fixture.

The five factors

Stage.

Where the task sits in the pipeline: emergency stop, active cut, confirmation, replanning, or background monitoring.

Skin.

How close the relevant tool path is to skin. Smaller clearance creates higher scheduling urgency.

Confirm.

Whether a human confirmation is pending and how long it has waited.

The five factors (Cont'd)

Safety.

The sandbox decision pressure: reject and stale-evidence handling outrank ordinary approved work.

Load.

Current queue or compute pressure for the robot path. Lower pressure gets a small preference to keep throughput stable.

Normalization contract

All five factors are integers in the closed range 0 to 1000. Do not use floating point in the contract function.

Factor	Example normalizer
Stage	emergency stop 1000, active cut 700, confirm 650, replan 450, background 150.
Skin	$\text{clamp}((12000 - d_{\mu m}) \times 1000 / 12000, 0, 1000)$.
Confirm	no pending confirm 0; pending confirm starts at 400 and rises with wait time.
Safety	reject or stale critical input 1000, confirm band 650, approve 100.
Load	queue pressure from 0 idle to 1000 saturated.

Pure priority function

Contract shape.

The function is pure: no clock reads, no queue reads, no mutation, no transport state. Inputs are already normalized.

```
fn priority(stage: i32, skin: i32, confirm: i32,  
            safety: i32, load: i32) -> i32 {  
    30 * stage  
+ 25 * skin  
+ 20 * confirm  
+ 20 * safety  
+ 5 * (1000 - load)  
}
```


Conformance fixture

Higher score wins. These rows are small enough to calculate by hand during cross-review.

Case	stage	skin	confirm	safety	load	score
A	800	900	100	1000	300	72000
B	600	400	1000	600	800	61000
C	400	100	0	200	100	23000

Expected order.

A before B before C. If Rust and TypeScript disagree on any row, the implementation is not conformant.

Preemption thresholds

Hard preemption.

A task with safety factor 1000, or a sandbox reject affecting current actuation, preempts immediately and may issue a stop path.

Soft preemption.

A candidate task may preempt the running task when its score is at least 12000 higher and it has waited at least 100 ms.

Commit window.

No soft preemption inside a sub-300 ms actuator commit window. Hard safety still wins.

Audit.

Record candidate score, running score, threshold, and reason.

Per-robot fairness

Keep the base score pure.

Fairness wraps the five-factor function. It does not change it.

Use per-robot credits.

Each robot queue earns credit up to a cap. Selection spends credit.

Effective ordering.

Rank by base priority plus credit, then wait age, then robot id.

Safety exception.

Fairness never delays hard safety preemption. Credits are for ordinary competition among eligible work.

Fairness wrapper sketch

```
fn pick_next(queues: &[RobotQueue], credit: &mut Credits) -> TaskId {  
    for r in queues { credit.add(r.id, 4000, 20000); }  
  
    let chosen = best_eligible_by(|t| (  
        t.base_priority + credit.get(t.robot_id),  
        t.wait_start_ms,  
        robot_order(t.robot_id),  
    ));  
  
    credit.spend(chosen.robot_id, 8000);  
    chosen.task_id  
}
```

What Ch5 and Ch6 must not drift on

Factor names and units.

A value named `skin` must mean the same normalized integer on both paths.

Integer arithmetic.

No floats, no language-specific rounding, no hidden precision differences.

Tie-breaks.

Oldest eligible task and canonical robot id must be specified, otherwise equal scores drift under load.

Fixture ownership.

If a team wants a new factor or weight, it is an ARB change, not a local edit.



Audit, Replay, Report

Why audit is part of safety

Safety claims need evidence.

It is not enough to say a plan was rejected. The log must show which inputs, thresholds, constraints, and code version produced that decision.

Replay needs determinism.

A recorded session should run through a later sandbox version and produce a comparable decision sequence.

Cross-version reports catch regression.

If a new version approves something the old version rejected, that is a safety-loosening event that requires review.

Audit-entry schema, minimum shape

```
{  
  "schema_version": "audit.v1",  
  "entry_id": "01HCR-C3-000001",  
  "time_unix_ms": 17600000000000,  
  "trace_id": "session-42/task-17",  
  "track": "desktop|web",  
  "robot_id": "robot-A",  
  "threshold_version": "c3.thresholds.v1",  
  "sandbox_input_hash": "sha256:..."  
}
```


Audit-entry schema (Cont'd)

```
{  
  "constraint_results": ["approve", "confirm", "reject"],  
  "decision": "approve|confirm|reject",  
  "priority_factors": {"stage": 800, "skin": 900},  
  "priority_score": 72000,  
  "code_version": "git:...",  
  "prev_hash": "sha256:...",  
  "entry_hash": "sha256:..."  
}
```

Audit storage responsibilities

Ch9 D-Bus.

Persist audit entries for the embedded Desktop path, including local broker behavior, PnP or reverse-proxy reachability, and SQLite storage if used.

Ch10 W-Bus.

Persist audit entries for the Web path, including MQTT-over-WSS, REST control events, and service-worker buffering if used.

Shared contract.

The stored schema and hash-chain meaning must match. Storage engines may differ; record semantics may not.

Replay algorithm

```
fn replay(log: AuditLog, sandbox: SandboxVersion) -> ReplayReport {  
    verify_hash_chain(&log)?;  
  
    for e in log.ordered() {  
        let input = load_by_hash(e.sandbox_input_hash)?;  
        compare(e.decision, sandbox.evaluate(input));  
    }  
  
    summarize_deltas()  
}
```

Replay rule.

Use recorded inputs, not reconstructed guesses. If an input hash cannot be resolved, the replay report is incomplete.

Cross-version safety report

Delta class	Example	Action
Same decision	reject stays reject	Count and archive.
Stricter	approve becomes confirm or reject	Review for false positives and operator load.
Looser	reject or confirm becomes approve	Block release until Safety and ARB review.
Reason changed	reject stays reject, cause changes	Review threshold or geometry drift.

Report key.

The report is about safety semantics, not benchmark speed. Performance belongs in a separate benchmark table.



Hotaru Named Access Points

Why Hotaru appears in C3

Safety decisions move through a runtime.

The sandbox, scheduler, audit store, UI, and robot path are connected by access points, not by direct function calls.

Transport swap should be boring.

A named access point lets the same logical endpoint bind to simulation or real transport by configuration.

The ABI is a contract surface.

Ch9 and Ch10 can implement different transports only if they preserve one handler shape and the same named semantics.

One handler shape, two roles

Inbound access point.

A server-role module processes one inbound unit per call, such as an approved-plan subscriber.

Outbound access point.

A client-role module produces one outbound unit per call, such as a twin publishing vision summary.

Shared shape.

Both roles use a request-context handler shape. The role changes binding and flow direction, not the user-written handler idea.

HCR payoff.

The same access point works in simulation and physical deployment.

Trait-signature draft

```
pub trait RequestContext: Send + Sized + 'static {  
    type Response; type Error;  
    fn role(&self) -> ProtocolRole;  
    fn into_response(self) -> Self::Response;  
}  
  
pub type Handler<C> = fn(C) -> HandlerFuture<C>;  
pub struct NamedAccessPoint<C> {  
    pub name: &'static str, pub role: ProtocolRole,  
    pub handler: Handler<C>,  
}
```


Channel and rebind rules

Idempotent close.

Calling `close` twice must be safe. Concurrent holders must not panic.

Peer-close-aware liveness.

`is_open` reflects both local closure and observed peer closure.

Multiplexing is hidden.

One MQTT or WebSocket connection may back many logical channels.

Rebind-safe references.

Rebinding a named path refreshes references, making hot-swap a runtime event.

Where the ABI touches this lecture

Sandbox.

The sandbox registers candidate-plan evaluation and emits decision records.

Scheduler.

The scheduler registers task intake, priority status, and preemption notices.

Audit.

The audit store consumes entries through one stable schema.

Robot path.

The robot subscribes only to approved plans; the name survives sim-to-real swap.



6

C3 Handoff

Team handoff checklist

Ch5 D-Brain and Ch6 W-Brain.

Implement the five-factor pure function, run the fixture, and document preemption plus fairness without modifying the base score.

Ch7 D-Safety and Ch8 W-Safety.

Implement the five constraints, the three-state decision rule, and fail-closed handling for malformed or stale input.

Ch9 D-Bus and Ch10 W-Bus.

Persist audit entries, support replay input lookup by hash, and preserve record meaning across storage differences.

All affected teams.

When in doubt, file an ARB clarification. Do not let one track become the accidental standard.

End-of-C3 verification

Constraint threshold agreement.

Ch7 and Ch8 compare the released threshold table line by line, including units, bands, freshness behavior, and worst-state ordering.

Priority fixture agreement.

Ch5 and Ch6 compare the fixture outputs exactly. The score must match, not just the ordering.

Audit schema agreement.

Ch9 and Ch10 confirm required fields, hash-chain behavior, and replay input lookup.

Hotaru ABI agreement.

Runtime and Bus teams agree on names, handler shape, role labels, channel liveness, and rebind semantics.

Takeaway

The sandbox decides.

The VLA proposes, the digital twin predicts, and the sandbox gates actuation through approve, reject, or confirm.

The scheduler is part of the safety story.

Priority is derived from stage, skin, confirm, safety, and load using a shared integer function.

The audit log makes claims replayable.

Every decision should be reproducible across versions from recorded input hashes and threshold versions.

Hotaru keeps the access point stable.

One handler shape and two roles let simulation and physical deployment share the same logical runtime surface.